Compte rendu du projet Palettiseur

Nathan Gaillet
Louis Chapuis

Contents

1 Introduction **3**

2 Démonstration fonctionnelle du palettiseur **3**

3 Architecture logicielle **3**
3.1 Décomposition fonctionnelle 3
3.2 Découpage en tâches parallèles 3
3.3 Stratégies d'activation des tâches 4
3.4 Communication et synchronisation inter-tâches 4

4 Implémentation avec FreeRTOS **4**

5 Gestion des ressources matérielles **5**
5.1 Utilisation du CPU 5
5.2 Utilisation de la mémoire 5

6 Consommation énergétique **6**
6.1 Fonctionnement à 48 MHz 6
6.2 Fonctionnement à 8 MHz 7
6.3 Fonctionnement à 8 MHz avec mode tickless 8
6.4 Analyse comparative 9

7 Conclusion **9**

1 Introduction

Ce projet a pour objectif la commande d'un palettiseur industriel à l'aide d'un microcontrôleur STM32 exécutant un système d'exploitation temps réel, FreeRTOS. Le système doit assurer un fonctionnement fiable et déterministe, tout en respectant des contraintes de ressources matérielles limitées (CPU, mémoire et consommation énergétique).

Ce compte rendu présente le fonctionnement global du palettiseur, l'architecture logicielle retenue, son implémentation sous FreeRTOS, ainsi qu'une analyse de l'utilisation des ressources matérielles et de la consommation énergétique.

2 Démonstration fonctionnelle du palettiseur

Le palettiseur est contrôlé par un programme temps réel permettant l'enchaînement automatique des différentes étapes nécessaires à la constitution d'une palette. Un cycle complet comprend la détection des cartons, leur prise en charge, leur déplacement et leur dépôt sur la palette.

3 Architecture logicielle

3.1 Décomposition fonctionnelle

Le fonctionnement global du palettiseur repose sur une chaîne fonctionnelle cohérente allant de l'acquisition des informations jusqu'à l'action sur l'environnement. Les fonctions d'acquisition des capteurs et de gestion des évènements ont été volontairement regroupées au sein d'une même entité fonctionnelle. Ce choix permet de centraliser l'évaluation de l'état du système et de simplifier la logique de déclenchement des transitions.

La décomposition fonctionnelle retenue est la suivante :

- Acquisition des trames de données, reconstruction et évaluation de l'état des capteurs, ainsi que génération des évènements associés.
- Gestion des différentes phases du processus de palettisation (arrivée des cartons, mise en forme, accumulation et palettisation).
- Transmission des commandes vers l'environnement de simulation.

Cette organisation permet de séparer clairement les responsabilités et de limiter les échanges inter-tâches inutiles tout en garantissant une vision globale cohérente de l'état du palettiseur.

3.2 Découpage en tâches parallèles

Sur la base de cette décomposition fonctionnelle, le système est implémenté à l'aide de plusieurs tâches FreeRTOS exécutées en parallèle :

- **Task_Read** : tâche centrale chargée de la lecture des capteurs et de la gestion des abonnements dans l'architecture publish/subscribe.
- **Task_Write** : tâche dédiée à la transmission des commandes vers Factory_IO.
- **Task1** : gestion de l'arrivée des cartons et mise en forme par groupes de deux à l'aide du bloqueur.
- **Task2** : gestion de la mise en place des cartons sur la porte et accumulation par groupes de six.
- **Task3** : gestion complète de la phase de palettisation, incluant l'arrivée de la palette, son déplacement à l'aide de l'ascenseur, la récupération des cartons par groupes de six, ainsi que l'acheminement final de la palette jusqu'à la sortie du palettiseur.

Chaque tâche correspond ainsi à une fonction identifiée lors de l'analyse fonctionnelle, garantissant une séparation claire des rôles et un parallélisme maîtrisé.

3.3 Stratégies d'activation des tâches

Les tâches sont activées en fonction de leur priorité et de leur rôle dans le fonctionnement global du système.

La tâche **Task_Read** est une tâche périodique. Elle est exécutée afin de surveiller l'état des capteurs et d'évaluer les conditions associées aux abonnements. Toutefois, cette analyse n'est réalisée qu'après la réception complète d'une trame de données, garantissant la cohérence des informations traitées. Ce fonctionnement permet de concilier une surveillance régulière du système avec une prise en compte fiable des données issues de la communication.

Les tâches **Task1**, **Task2** et **Task3** sont principalement événementielles. Elles restent bloquées tant que les conditions nécessaires à leur activation ne sont pas satisfaites, et sont débloquées à la réception des évènements correspondant aux abonnements qu'elles ont définis.

La tâche **Task_Write** est activée lorsqu'une commande est disponible à transmettre vers Factory_IO. Son exécution est découplée des tâches de décision afin d'éviter tout impact sur le comportement temps réel du système.

L'ordonnancement des tâches repose sur les mécanismes de priorité fournis par FreeRTOS, permettant de garantir la réactivité du système et le respect des contraintes temporelles.

3.4 Communication et synchronisation inter-tâches

La communication inter-tâches repose sur une architecture de type *publish/subscribe*. Les tâches de gestion publient des abonnements décrivant les conditions d'activation attendues, tandis que la tâche **Task_Read** publie les évènements lorsque l'état des capteurs satisfait ces conditions.

Ce mécanisme permet un découplage fort entre l'acquisition des capteurs et la logique de décision, tout en autorisant la combinaison de plusieurs capteurs pour déclencher une transition d'état.

L'envoi des commandes vers Factory_IO est assuré par la tâche **Task_Write**. Les échanges entre tâches s'appuient sur les mécanismes de communication et de synchronisation proposés par FreeRTOS (queues et notifications), garantissant une exécution déterministe et cohérente du cycle de palettisation.

4 Implémentation avec FreeRTOS

L'implémentation logicielle s'appuie sur la modélisation architecturale précédente. Les transitions entre états sont réalisées exclusivement via le mécanisme publish/subscribe.

Les commandes associées à chaque état sont transmises à la scène par une tâche dédiée utilisant l'UART avec DMA, ce qui permet de limiter la charge CPU.

Une capture d'un cycle complet du palettiseur a été réalisée à l'aide de l'outil Tracealyzer en mode streaming. Le schéma fonctionnel du programme, extrait depuis la vue *Communication Flow*, est présenté ci-dessous.

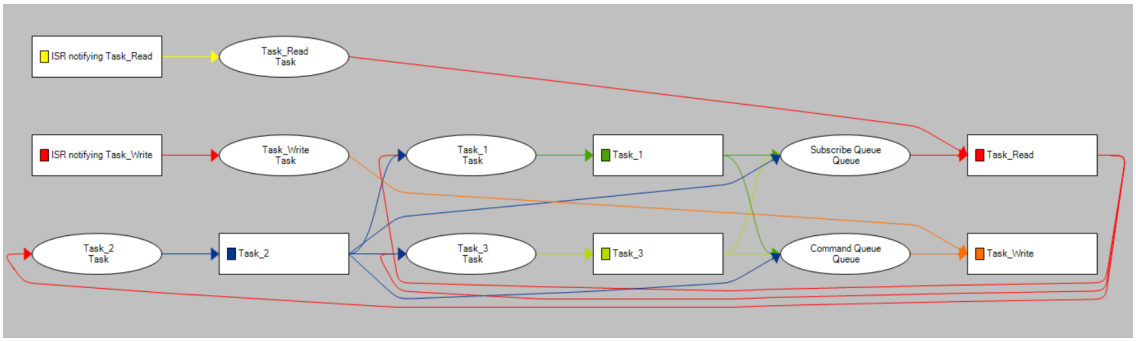

Figure 1: Schéma fonctionnel extrait de Tracealyzer

5 Gestion des ressources matérielles

5.1 Utilisation du CPU

L'occupation du processeur a été analysée à l'aide de l'outil Tracealyzer, permettant d'observer la répartition du temps d'exécution entre les différentes tâches du système.

Dans une première configuration, le projet a été développé avec une fréquence CPU de 48 MHz. Les mesures effectuées montrent que le processeur passe environ 99.943 % de son temps dans la tâche *Idle*, ce qui indique une charge CPU extrêmement faible. Ce résultat met en évidence que la fréquence initialement choisie est largement surdimensionnée par rapport aux besoins réels de l'application.

Afin d'évaluer l'impact d'une réduction de la fréquence, le système a ensuite été configuré pour fonctionner à 8 MHz. Dans cette configuration, le temps passé dans la tâche *Idle* reste très majoritaire, avec une valeur de 97.647 %. La tâche *TzCtrl*, associée à l'outil de traçage, représente quant à elle environ 1.997 % du temps processeur.

Ces résultats montrent que, malgré la diminution significative de la fréquence CPU, le système conserve une large marge de calcul.

Cette analyse met en évidence qu'une fréquence de fonctionnement plus faible est suffisante pour répondre aux contraintes temps réel du projet, tout en permettant une réduction de la consommation énergétique du système.

Actor	Count	CPU Usage
IDLE	1	99.943
Task_2	59	0.007
TzCtrl	3283	0.000
Task_1	189	0.012
Task_3	23	0.002
Tmr Svc	1	0.000
Task_Write	241	0.012
Task_Read	1811	0.023
ISR notifying Task_Read	326	0.000
ISR notifying Task_Write	130	0.000

(a) Occupation CPU à 48 MHz

Actor	Count	CPU Usage
IDLE	1	97.647
Task_2	30	0.043
TzCtrl	1760	1.997
Task_1	100	0.095
Task_3	11	0.010
Tmr Svc	1	0.000
Task_Write	133	0.056
Task_Read	969	0.152
ISR notifying Task_Read	167	0.000
ISR notifying Task_Write	68	0.000

(b) Occupation CPU à 8 MHz

Figure 2: Comparaison de l'occupation CPU mesurée avec Tracealyzer pour deux fréquences différentes

5.2 Utilisation de la mémoire

L'analyse de l'utilisation mémoire s'est concentrée principalement sur le dimensionnement des piles (*stack*) associées aux différentes tâches FreeRTOS. Chaque tâche a été créée avec une pile de 128 mots, dimensionnement choisi de manière conservatrice afin de garantir un fonctionnement fiable du système.

L'outil Tracealyzer a été utilisé pour analyser l'occupation réelle des piles au cours de l'exécution. Les résultats montrent que, pour l'ensemble des tâches, une marge de pile non utilisée subsiste, ce qui confirme l'absence de risque de débordement de pile et valide le dimensionnement retenu.

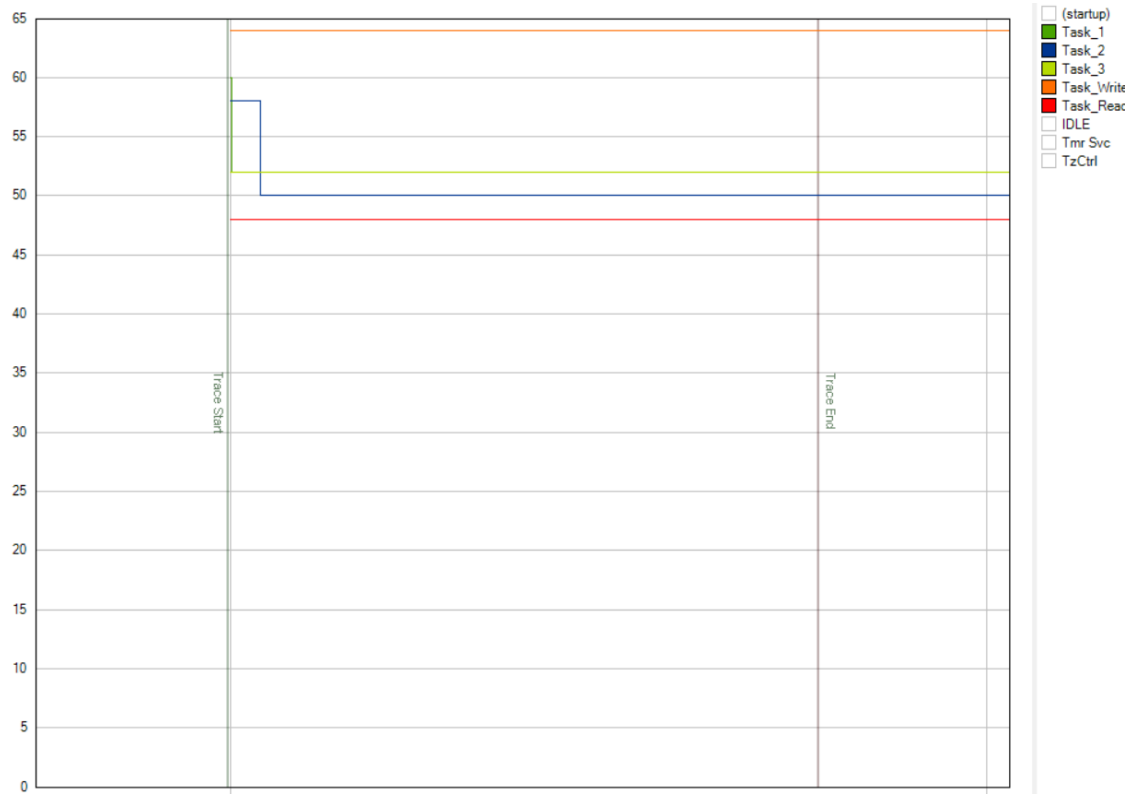

Figure 3: Utilisation des piles des tâches (Stack usage) mesurée avec Tracealyzer

6 Consommation énergétique

La consommation énergétique du système a été analysée à travers trois configurations différentes, afin d'évaluer l'impact de la fréquence CPU et de l'utilisation des mécanismes basse consommation de FreeRTOS.

6.1 Fonctionnement à 48 MHz

Dans une première configuration, le microcontrôleur fonctionne à une fréquence CPU de 48 MHz. La consommation mesurée dans cette configuration est d'environ 14 mA. Cette valeur constitue une référence, correspondant au fonctionnement nominal initial du projet.

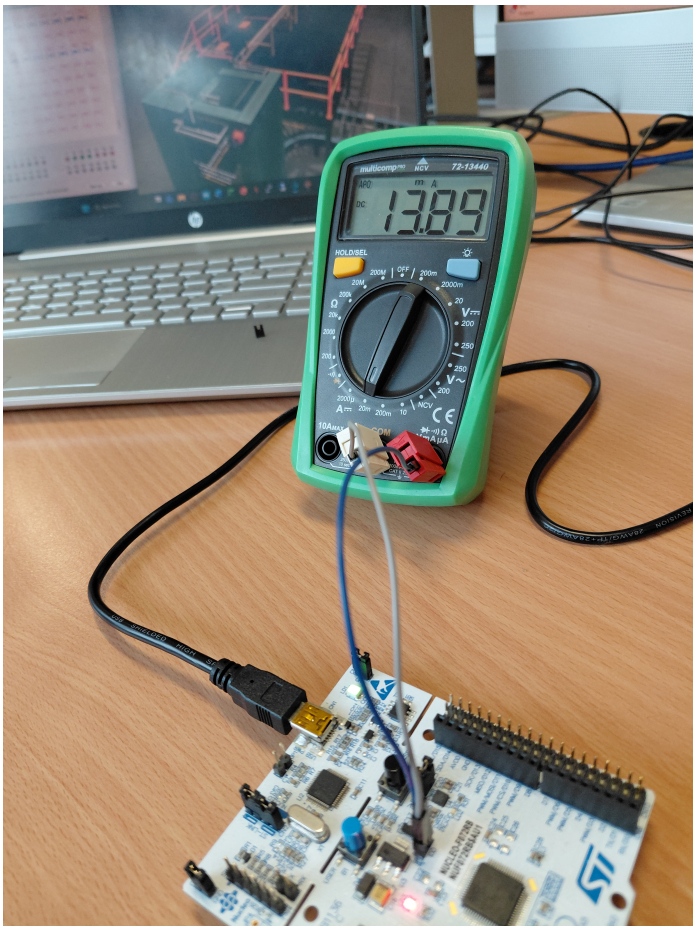

Figure 4: Mesure de la consommation du système à 48 MHz

6.2 Fonctionnement à 8 MHz

Dans une seconde configuration, la fréquence CPU a été réduite à 8 MHz, sans modification du reste de l'architecture logicielle. La consommation mesurée chute alors à environ 4 mA. Cette diminution significative confirme que la fréquence initiale de 48 MHz était largement surdimensionnée par rapport aux besoins réels du système.

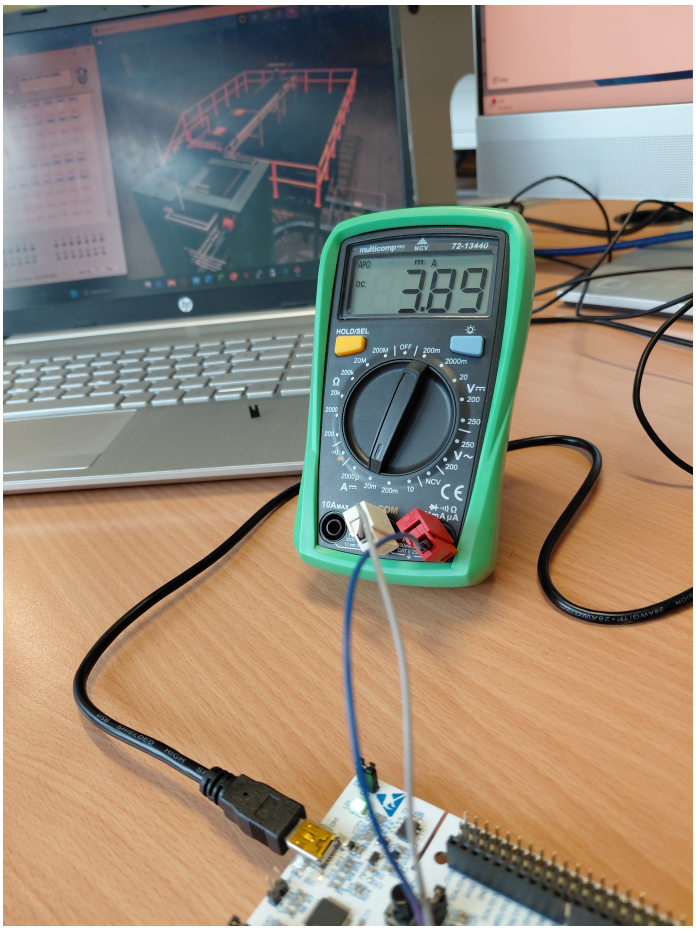

Figure 5: Mesure de la consommation du système à 8 MHz

6.3 Fonctionnement à 8 MHz avec mode tickless

Enfin, une troisième configuration combine une fréquence CPU de 8 MHz avec l'activation du mode *tickless* de FreeRTOS. Ce mode permet de supprimer les interruptions périodiques du système lorsque celui-ci est inactif, favorisant l'entrée en modes basse consommation.

Dans cette configuration, la consommation mesurée est d'environ 3 mA. Cette optimisation permet donc de réduire encore la consommation énergétique, sans impact sur le fonctionnement fonctionnel du palettiseur.

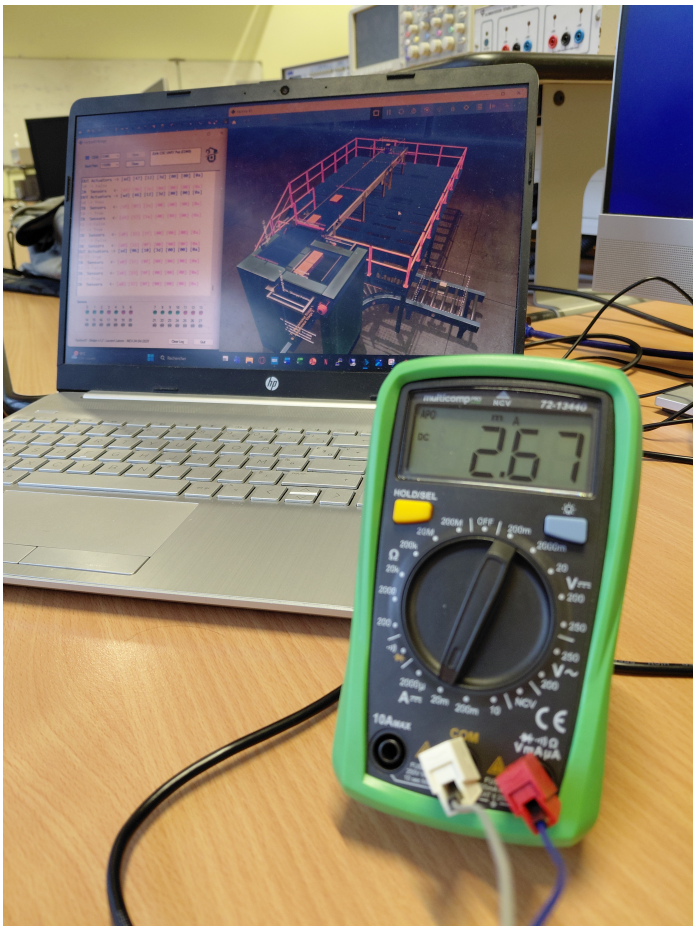

Figure 6: Mesure de la consommation du système à 8 MHz avec mode tickless

6.4 Analyse comparative

Les résultats obtenus mettent en évidence une réduction significative de la consommation énergétique du système :

- une division par plus de trois de la consommation en passant de 48 MHz à 8 MHz,
- un gain supplémentaire grâce à l'activation du mode tickless, permettant de réduire encore le courant consommé.

Ces résultats montrent qu'il est possible d'optimiser efficacement la consommation énergétique du palettiseur en adaptant la fréquence CPU et en exploitant les mécanismes basse consommation offerts par FreeRTOS, tout en conservant un fonctionnement temps réel conforme aux exigences du projet.

7 Conclusion

Ce projet a permis de mettre en œuvre un système temps réel complet basé sur FreeRTOS pour la commande d'un palettiseur. L'architecture retenue s'est révélée robuste et évolutive, tout en respectant les contraintes de ressources matérielles imposées.

Les analyses de performance, de mémoire et de consommation énergétique ont permis d'optimiser le système et d'approfondir la compréhension des problématiques liées aux systèmes embarqués temps réel.